The GNUnet Peer-to-Peer Framework

Christian Grothoff
christian@grothoff.org

https://gnunet.org/

Agenda

- File-Sharing in GNUnet (Christian Grothoff)
- Transport Selection in GNUnet (Matthias Wachs)
- Non-anonymous Routing in GNUnet (Nate Evans)

What is GNUnet?

GNUnet is a peer-to-peer framework:

- Focus is on "security"
- Most components can be re-used:
  - Authentication
  - Peer discovery
  - Encrypted channels
  - Accounting and resource allocation
- extensible

Outline of the Talk

1. Using economics against the tragedy of the commons
2. Encoding data for censorship-resistant file-sharing
3. Achieving anonymity
4. Current & future work

Accounting in GNUnet

- Goals & requirements
- Human relationships!
- The excess-based economy
- Transitivity
- Open issues

Goals

- Compatible with anonymous participation
- Reward contributing nodes with better service
- Detect attacks and abuse:
  - Flooding with content
  - Excessive free-loading
- Allow *harmless* amounts of free-loading
- Performance, no overpricing

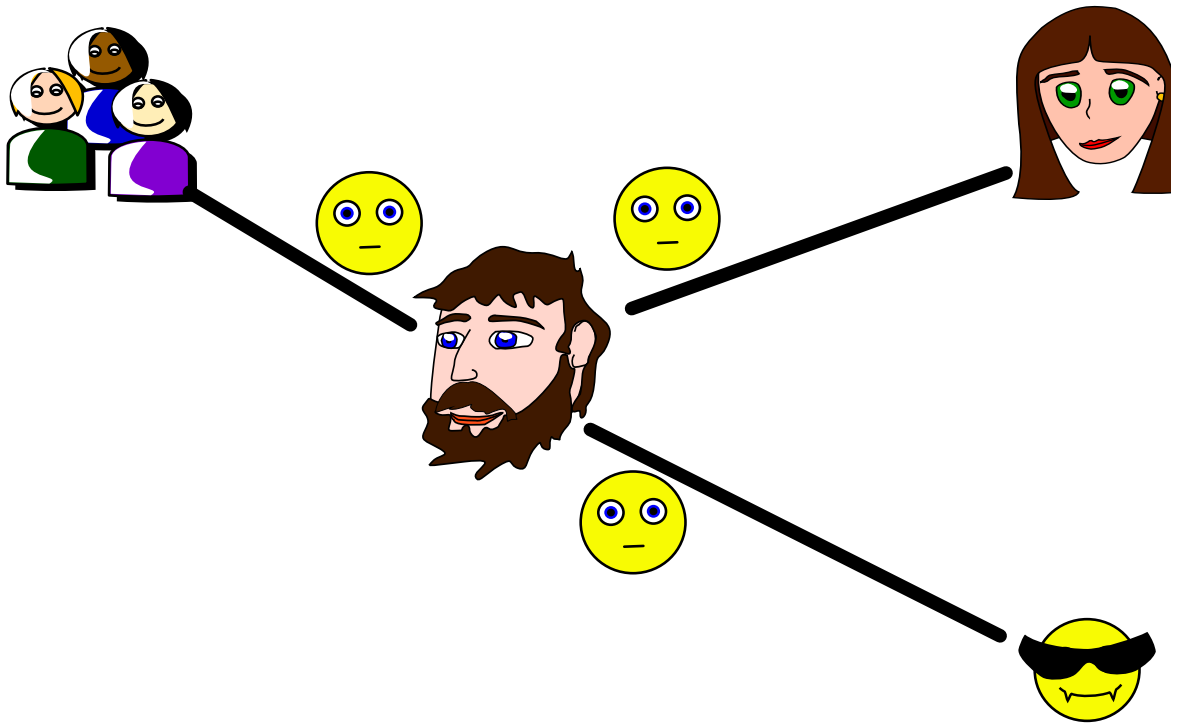
Requirements & Assumptions

- No central server or trusted authority
  - Rules out designs using electronic cash!
  - Causes problem of initial accumulation (see [2, Engels, 1844])
- Everybody else is malicious and violates the protocols
- Everybody can make-up a new identity at any time
- New nodes should be able to join the network

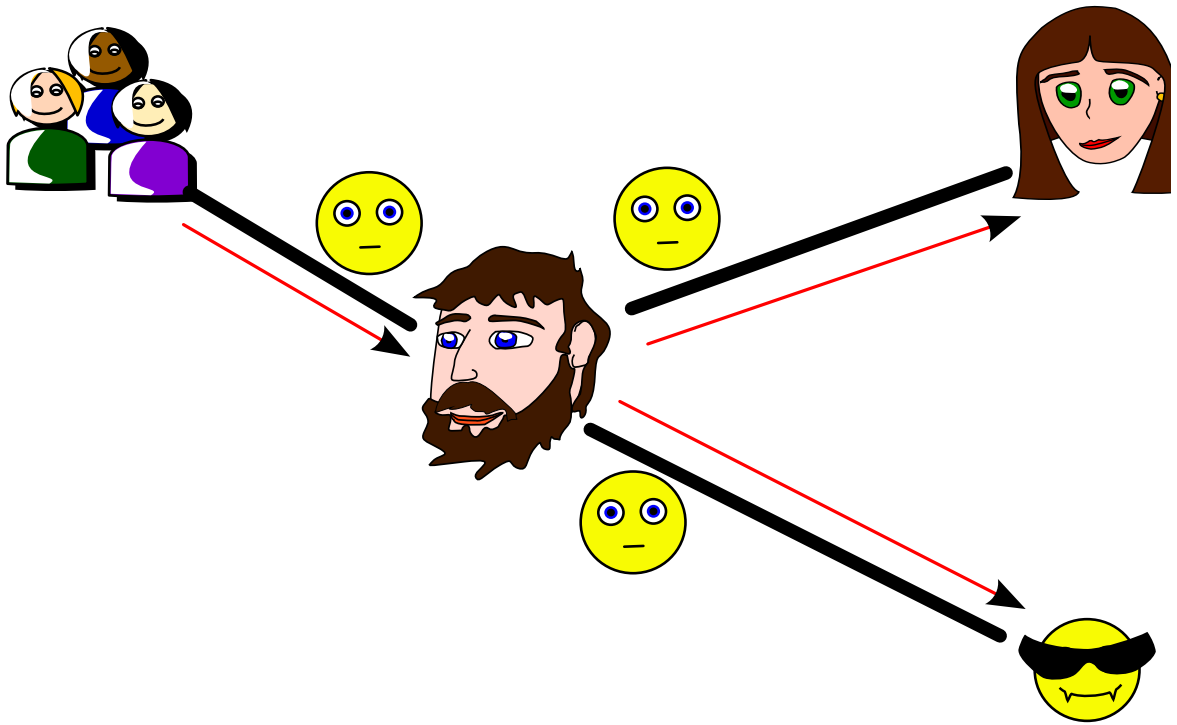
Human Relationships

- Given verifiable interactions
  $\Rightarrow$ No trusted authority needed to form trust
- Opinions are formed on a one-on-one basis, and
- may not be perceived equally by both parties
- We do not charge for every little favour
- We are grateful for every favour
- There are no guarantees in life. In particular, Alice does not have to be kind to Bob just because he was kind to her.

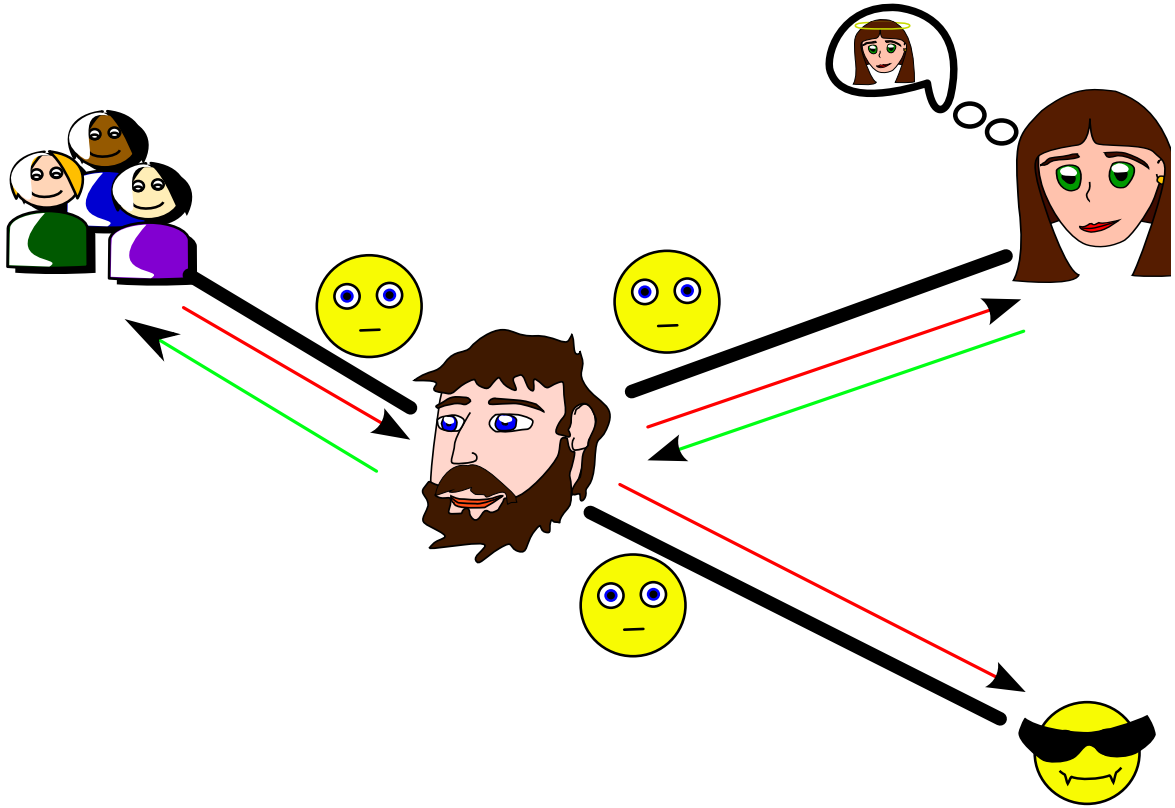
Excess-based Economy Illustrated (1/8)



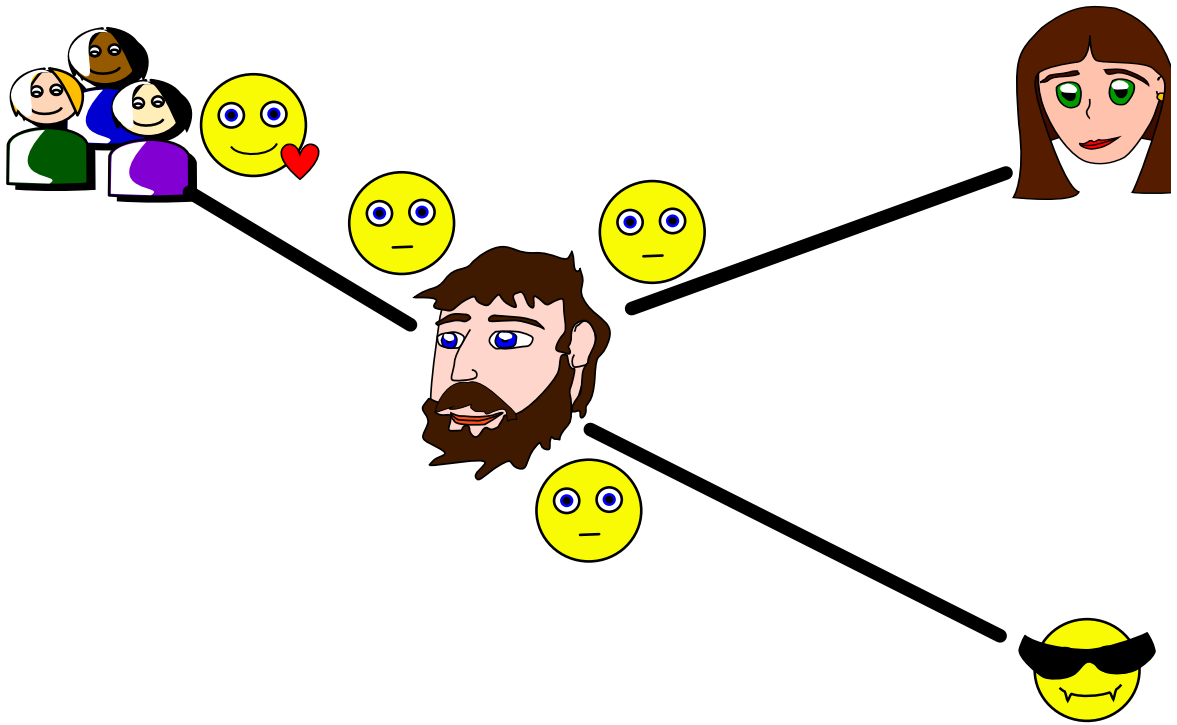
Excess-based Economy Illustrated (2/8)



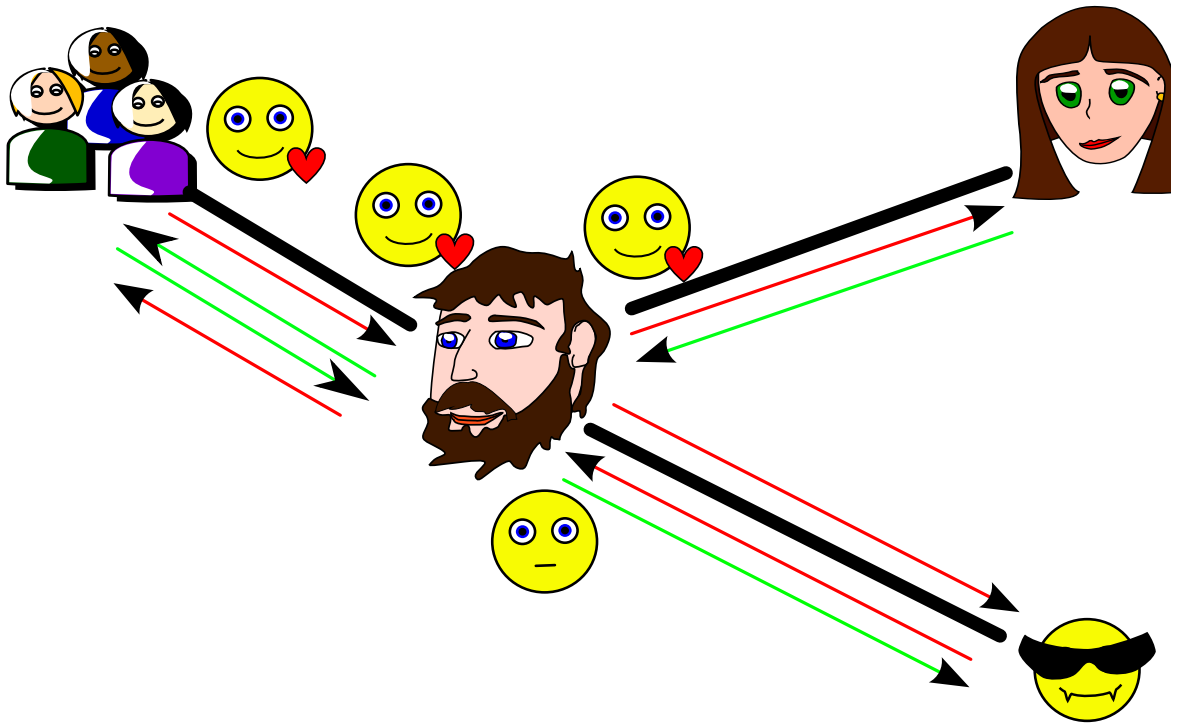
Excess-based Economy Illustrated (3/8)



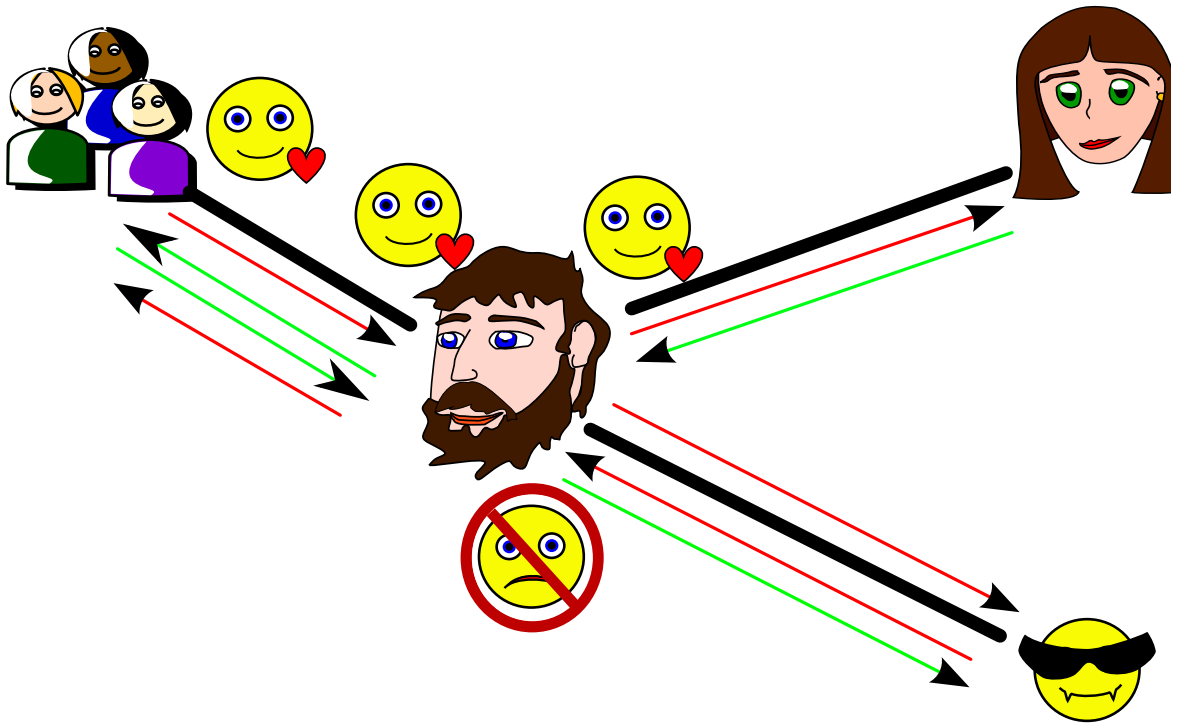
Excess-based Economy Illustrated (4/8)



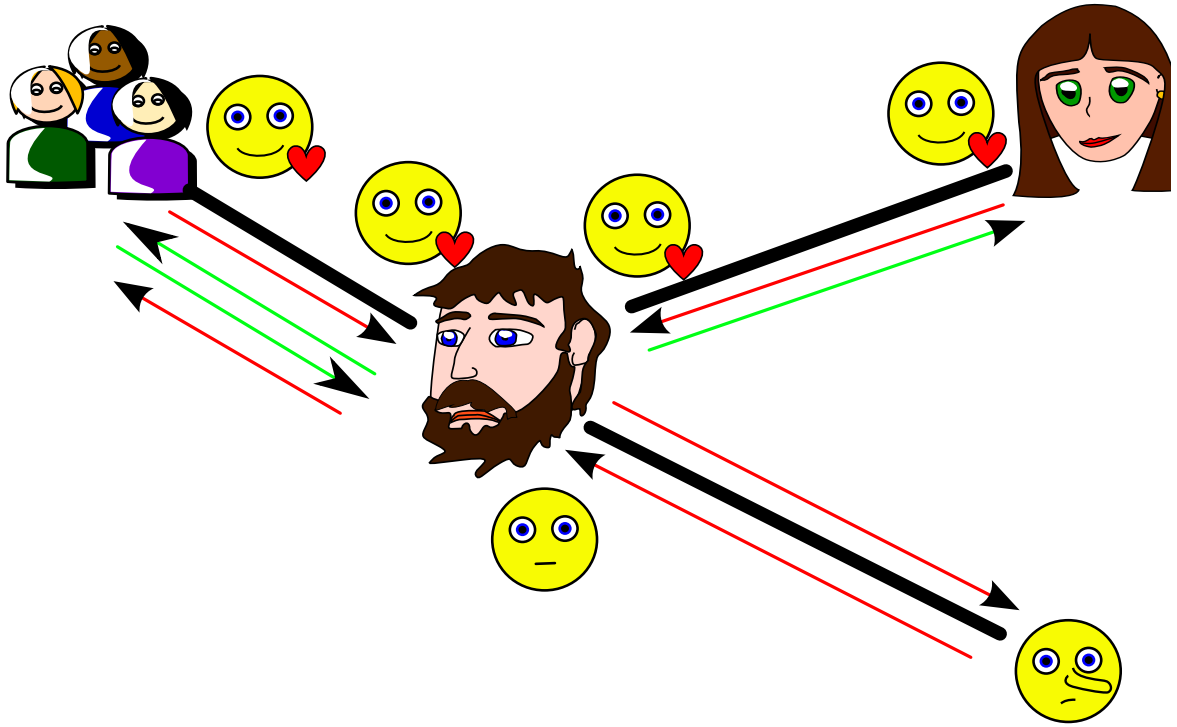
Excess-based Economy Illustrated (5/8)



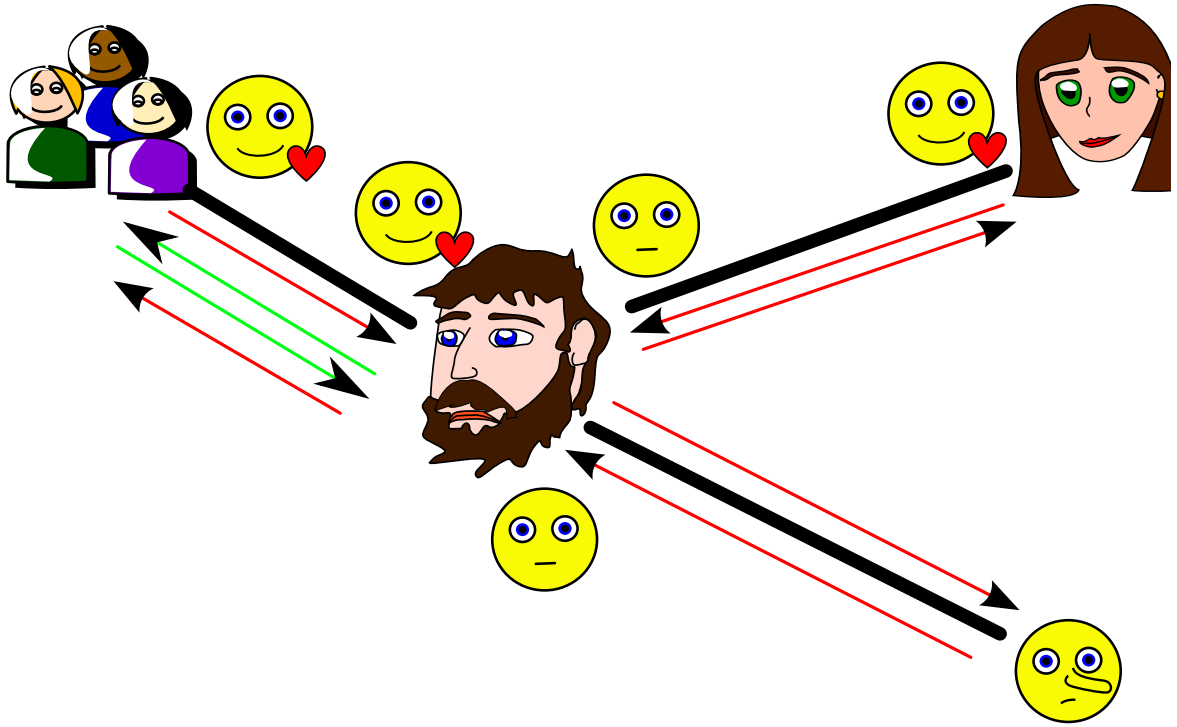
Excess-based Economy Illustrated (6/8)



Excess-based Economy Illustrated (7/8)

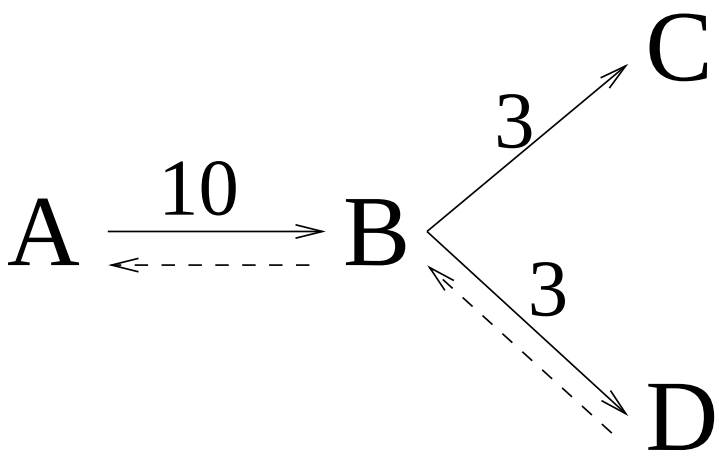


Excess-based Economy Illustrated (8/8)



Transitivity

If a node acts on behalf on another, it must ensure that the sum of the charges it may suffer from other nodes is lower than the amount it charged the sender:



Transitivity in the GNUnet economy.

Excess-based Economy: Summary

GNUnet's economy is based on the following principals:

- If $idle$, doing a favour for free does not cost anything;
- If somebody does you a favour, remember it;
- If $busy$, work for whoever you like most, but remember that you paid the favour back;
- Have a $neutral$ attitude towards new entities;
- Never dislike anybody (they could create a new identity anytime).

Economy: Conclusion

+ Preserves sender and receiver anonymity
+ Rewards users that contribute resources with better service
+ Lightweight, distributed mechanism
- Requires excess resources to bootstrap economy

Details in [3].

Open research question: How to best set prices?

Economy: Requirements for Encoding

Key premise: "verifiable interactions"

$\Rightarrow$ Need content encoding that makes cheating not viable!

Encoding Data for File-Sharing

- Requirements
- Content encoding
- Support for searching

Problems with Other Encoding Mechanisms

- Content distributed in plaintext (e.g. gnutella) facilitates censorship and may void deniability
- Content must be inserted into the network and is then stored twice, in plaintext (by the originator) and encrypted (by the network – e.g. Freenet)
- Independent insertions of the same file result in different copies in the network (e.g. Publius [4])
- Verification of content integrity can only occur after download is complete (most early P2P systems)

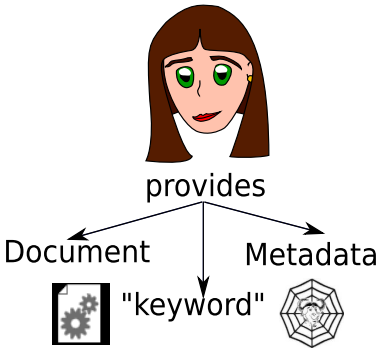
Properties of ECRS Encoding

- Breaks large files into small, uniform blocks
- Keeps storage (and bandwidth) overhead small
- Intermediaries $cannot$ view content or queries
  $\Rightarrow$ Peers can send replies to queries and plausibly deny having knowledge of their contents
- Intermediaries are able to verify validity of responses
  $\Rightarrow$ Enables swarming, even in the presence of malicious peers trying to corrupt files

Properties of ECRS Implementation

- All operations performed by routers have expected runtime and memory use of $O(1)$
- All operations performed by responders have expected runtime $O(\log n)$ where n is the size of the datastore (under the assumption that a modern database with index can do lookups in $O(\log n)$)
- All receiver operations have (amortized) runtime $O(n)$ where n is the size of the result set or the size of the file; memory use for files of size n can be $O(\log n)$ with a small constant

ECRS Illustrated (1/9)



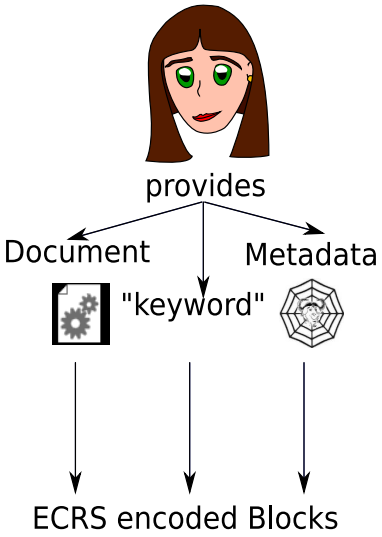
How to get the Keywords?

- Automatically extract metadata!
- Many file formats
  $\Rightarrow$ pluggable architecture

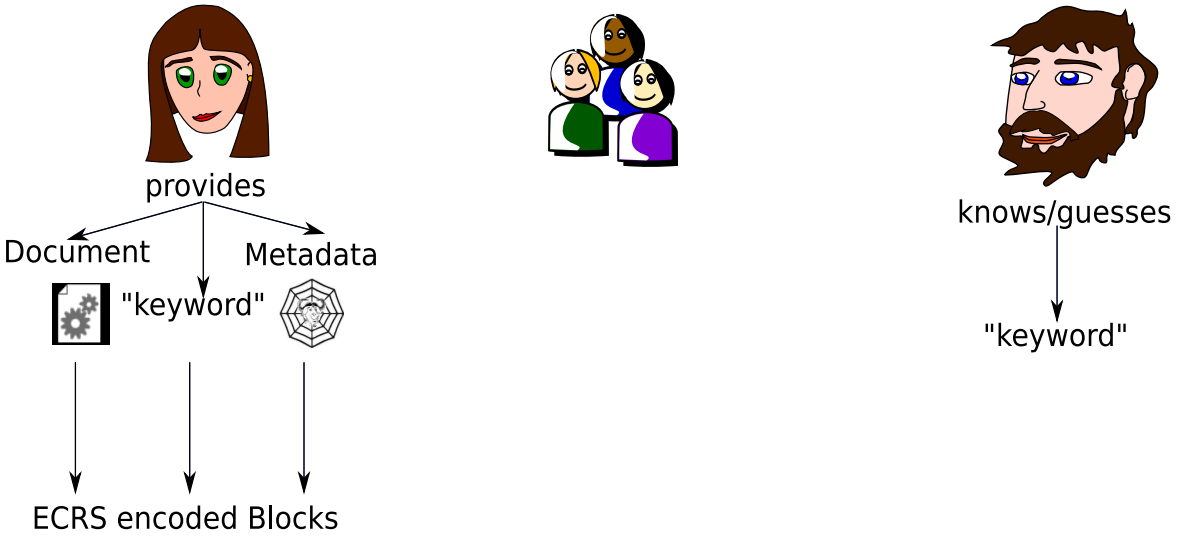
We created GNU libextractor for this:

`http://www.gnu.org/software/libextractor/`

ECRS Illustrated (2/9)

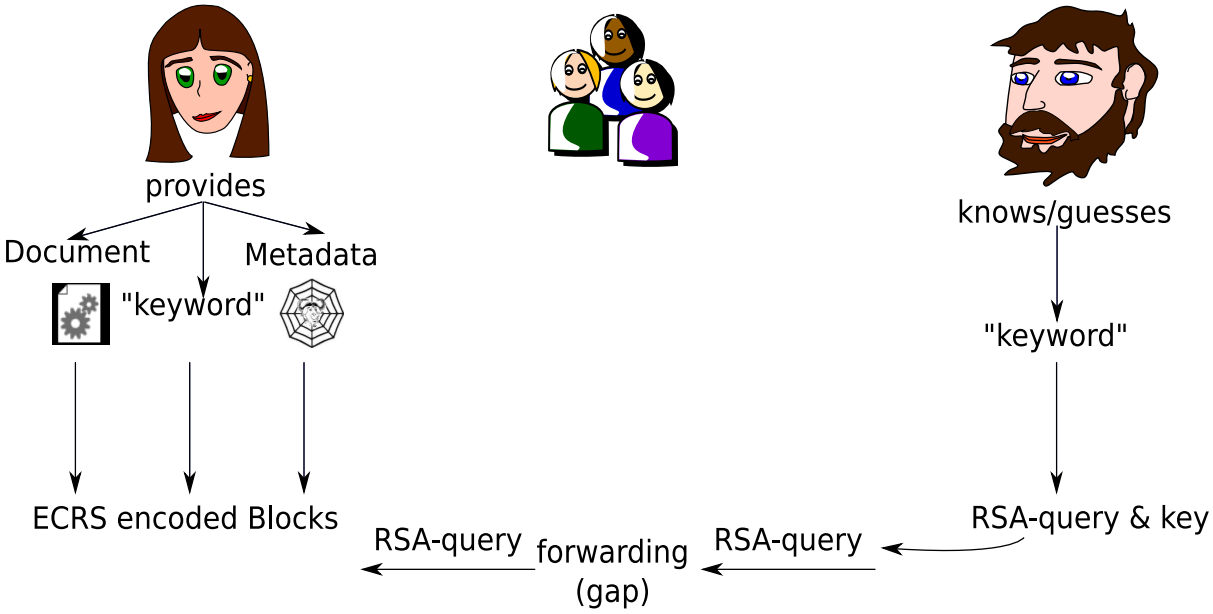


ECRS Illustrated (3/9)

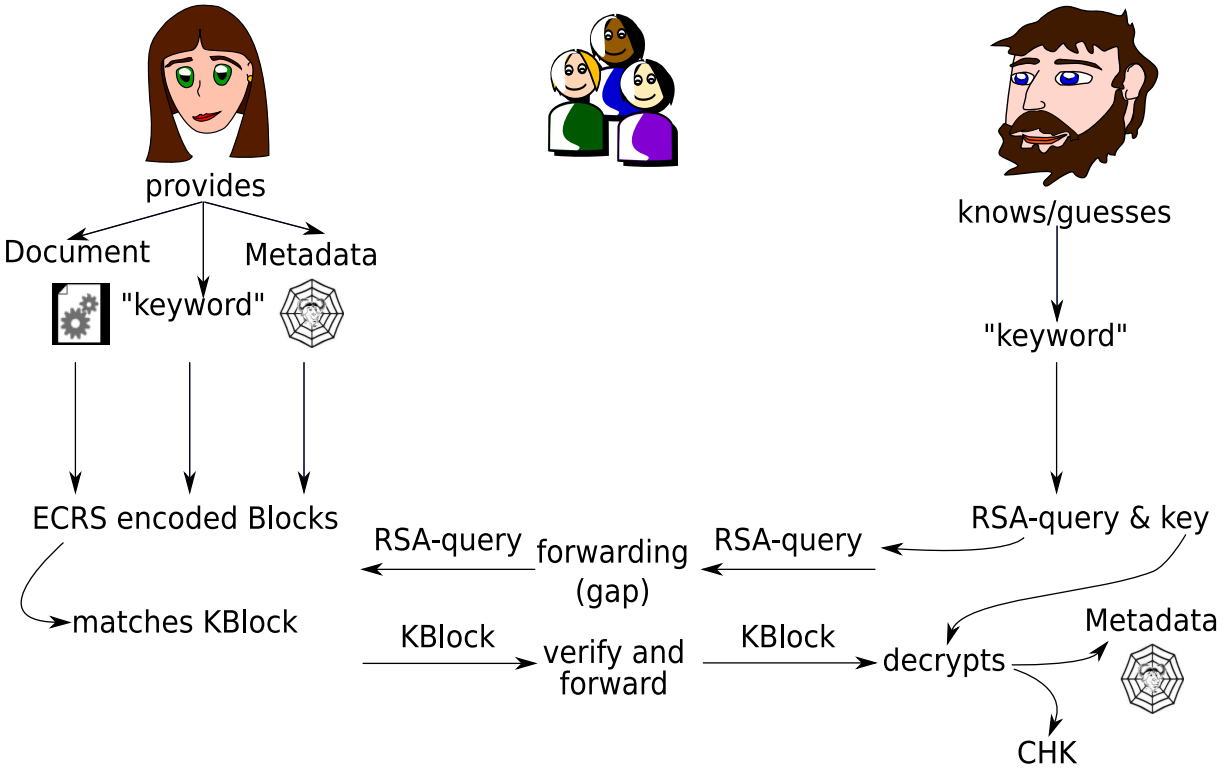


ECRS Illustrated (4/9)

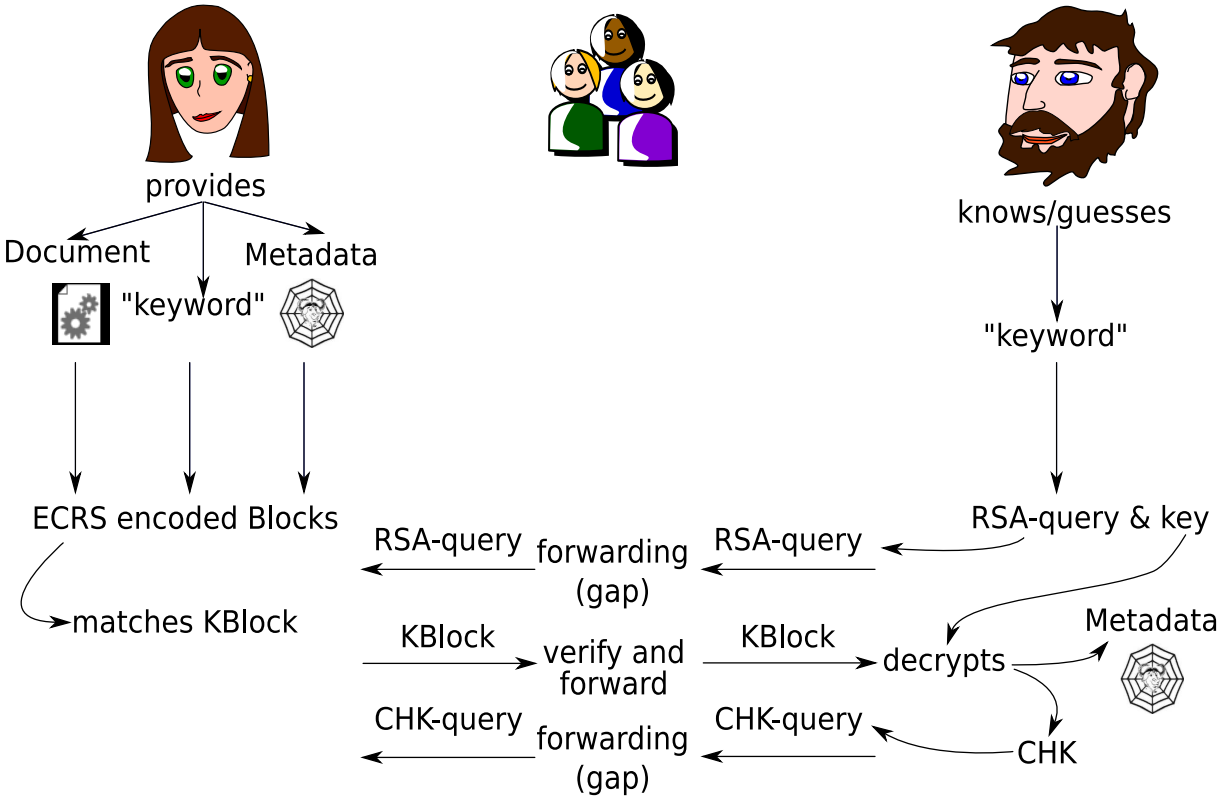
ECRS Illustrated (5/9)



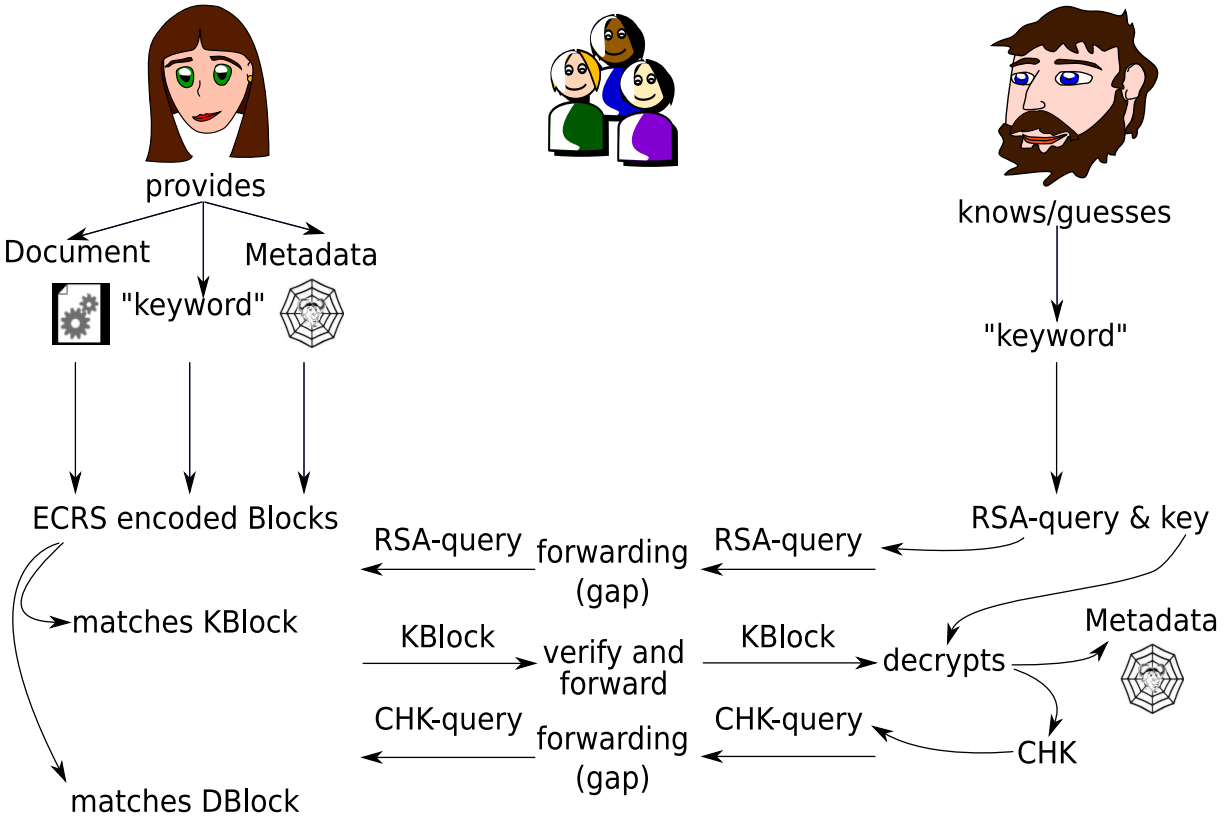
ECRS Illustrated (6/9)



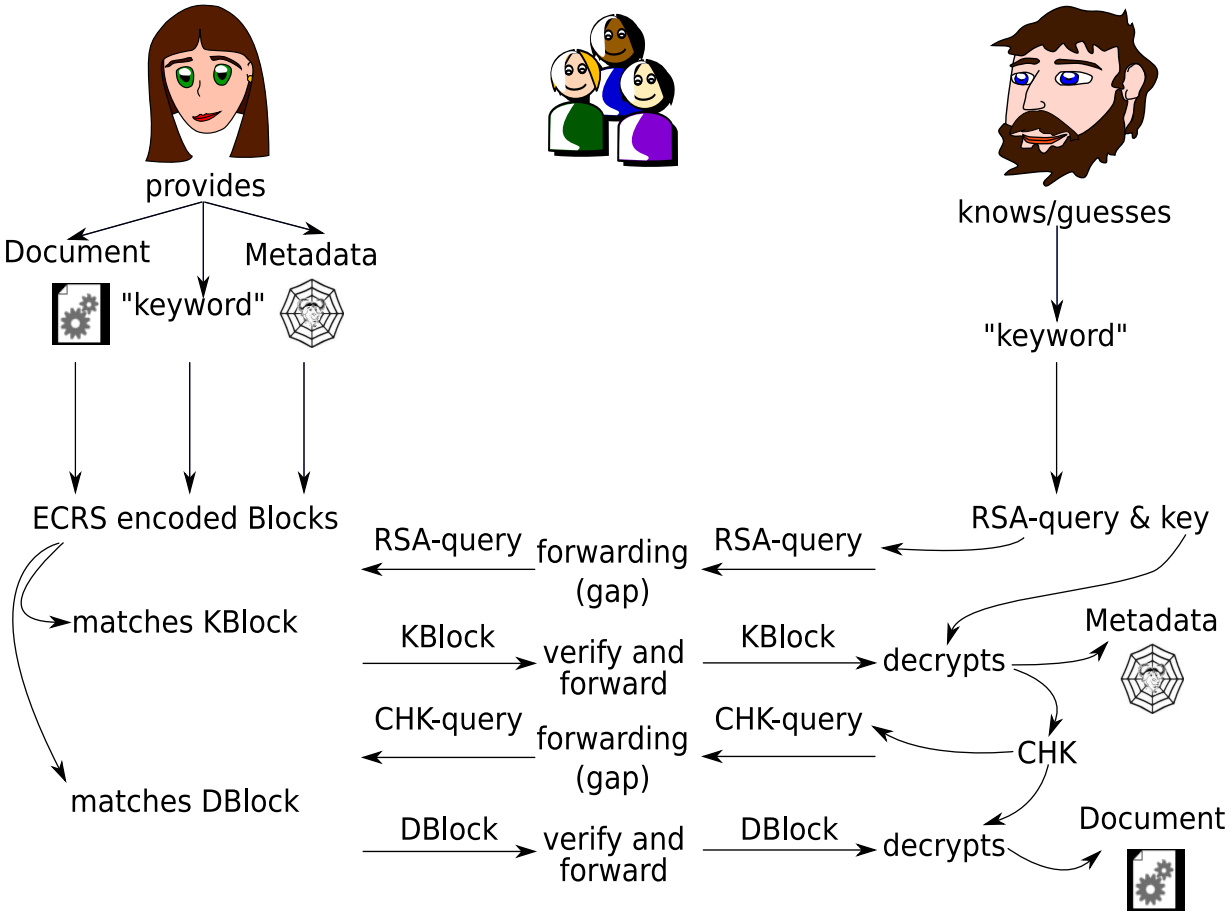
ECRS Illustrated (7/9)



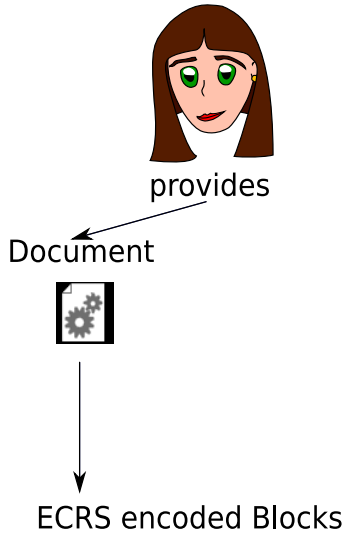
ECRS Illustrated (8/9)



ECRS Illustrated (9/9)

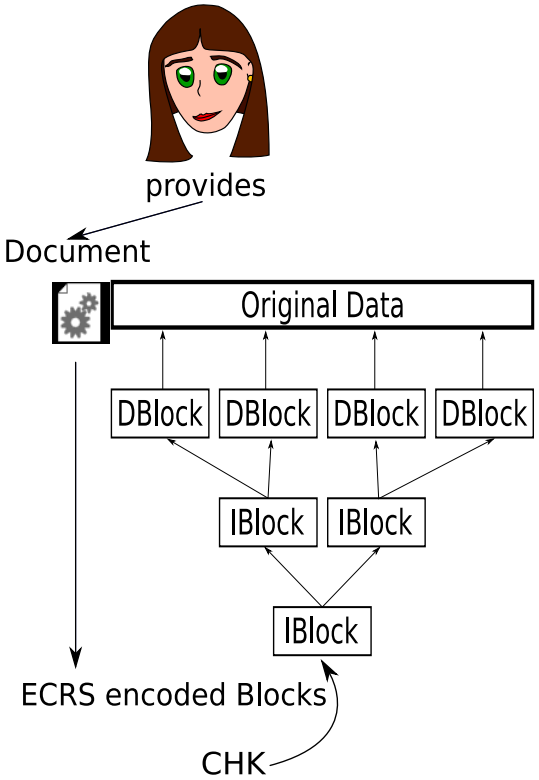


ECRS Details: Document Encoding (1/3)



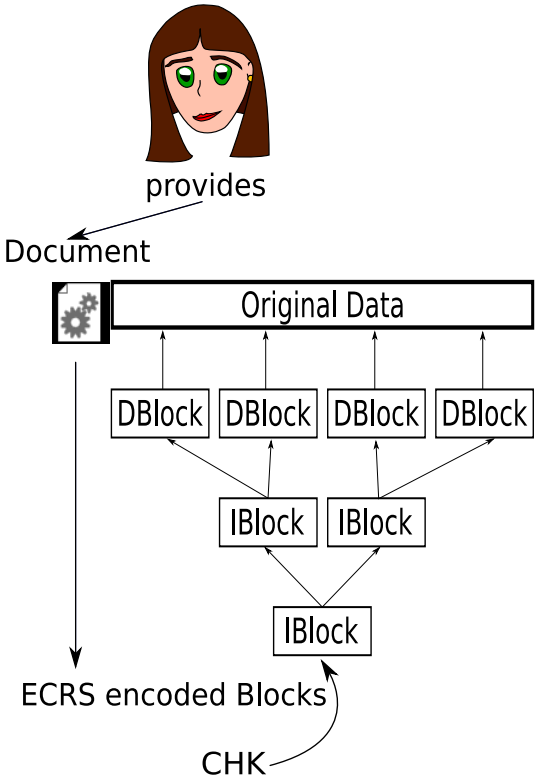
- Split content into 32k blocks B
- AES-256 encrypt B with $H(B)$
- Store $E_{H(B)}(B)$ under $H(E_{H(B)}(B))$

ECRS Details: Document Encoding (2/3)



- Split content into 32k blocks B
- AES-256 encrypt B with $H(B)$
- Store $E_{H(B)}(B)$ under $H(E_{H(B)}(B))$
- Build tree containing up to 256 CHK pairs: $H(B), H(E_{H(B)}(B))$

ECRS Details: Document Encoding (3/3)

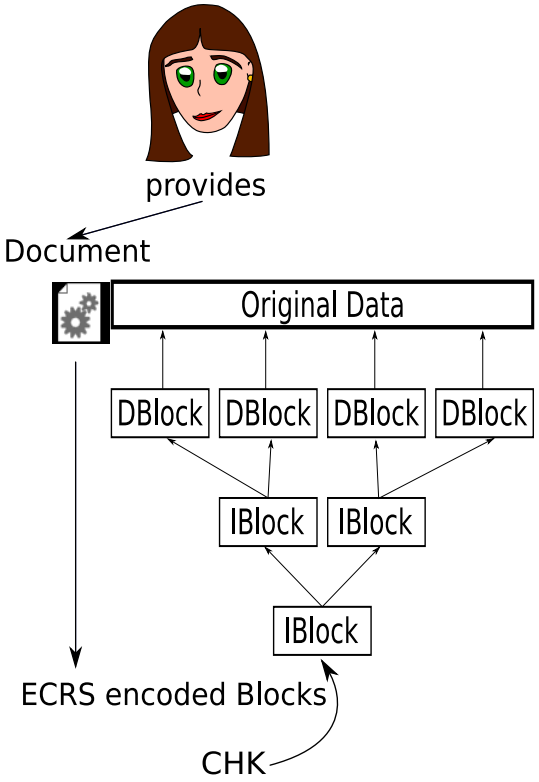


- Encryption of blocks independent of each other
- Inherent integrity checks
- Independent reproduction yields same encrypted blocks
- Small blocksize makes traffic more uniform
  $\Rightarrow$ traffic analysis is harder

ECRS Details: Document Encoding Limitations

- If the exact data can be guessed... participating hosts can match the content.
  Intended to reduce storage costs!

ECRS Search Design Requirements



- Retrieve content with simple, natural-language keyword
- Guard against malicious hosts: prevent attackers from providing useless replies!
- Do not expose actual keyword used
- Do not expose CHK or metadata: encrypt CHK and metadata as well!

ECRS Searching: KBlocks

Let R be the (plaintext) metadata and CHK.

- For each keyword K use K to generate RSA key pair $(PRIV_K, PUB_K)$, store $E_{H(K)}(R), PUB_K$ signed with $PRIV_K$
- User searching also computes RSA key pair and sends query: $H(PUB_K)$
- Intermediates match PUB_K against $H(PUB_K)$ and verify signature

Benefits and Limitations of KBlocks

+ Malicious peer cannot learn R without guessing the keyword

+ Malicious peer must guess keyword to generate valid reply

+ Malicious peer cannot modify reply without being detected

- Cryptographic operations are quite expensive

Open Issues

- Multiple Search Results
- Content updates
- Approximate queries

The Multiple Search Result Problem

- Responder can not send "fake" response (ECRS)
- Responder can send same response again and again

$\Rightarrow$ No incentive to look for alternative responses!

$\Rightarrow$ First (few) responses to keyword spread far and wide, others will never be displayed!

$\Rightarrow$ Need to use creative keywords (but in that case, caching is much less effective!)

Solution (1/2)

- As part of the query, communicate what replies are not acceptable
- Can not include full replies (too big)

$\Rightarrow$ Use bloomfilter of hash codes of encrypted replies

Solution (2/2)

- Bloomfilter is probabilistic
- Even relatively generous bloomfilters would filter approximately $1{:}2^{10}$ valid replies
- Solution: add random 32-bit nonce to hash function, change nonce (sometimes) when repeating requests

$\Rightarrow$ False-positives less than $1{:}2^{42}$

Other Solutions to Search

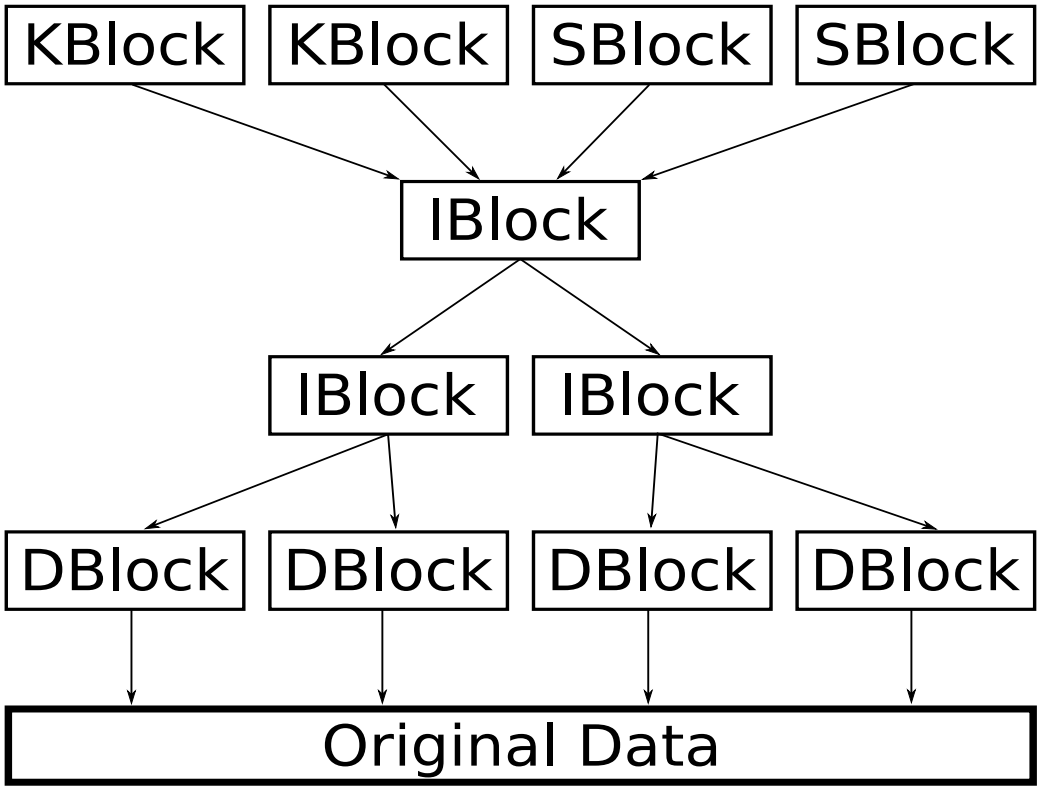
- Directories
- Pseudonyms:
  - `SBlocks` are essentially pseudonym-signed `KBlocks` (with possibly some additional information, such as pointers to updates)
  - We use the term **namespace** to refer to all content signed by the same pseudonym

How to Name Pseudonyms

- For security, only the public key can do
- Public key (or hash of public key) is not user-friendly!
- Owner could provide description
- Description may not be unique in the network!

$\Rightarrow$ Make descriptions unique for $each\ peer$ by adding description collision sequence number

Content Encoding: Summary



More details at `https://gnunet.org/ecrs` ([1]).

Open Issues

- Approximate queries
- Reputation management for pseudonyms

Anonymity in GNUnet: Motivation

- Real world: whistle blowers, political speech, human rights work, ...
  $\Rightarrow$ need usable system, not just design
- Real world adversary:
  - Can observe all the network links
  - Can modify, delete and inject messages
  - Can participate and pretend to represent normal users
- Real world users:
  - Impatient, want to manage significant amounts of data
  - Often not in need for strong protections

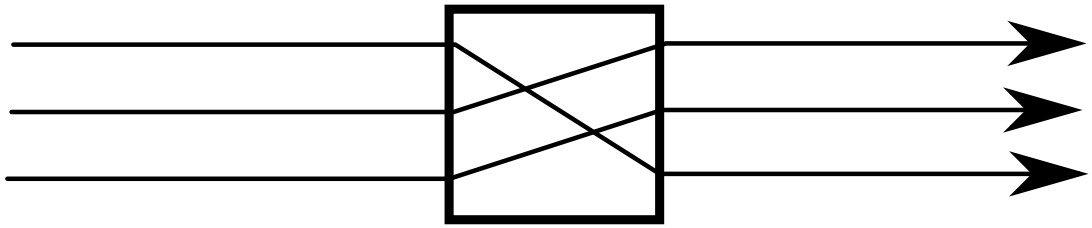
$\Rightarrow$ need to be able to trade-off anonymity/performance

Anonymous File-sharing: Goals

- Sender and receiver anonymity
- User should be able to decide how much anonymity is needed
- User should not rely on other peers' resources for anonymity
- Each user should pay the cost for his own anonymity requirements

The Traditional Approach

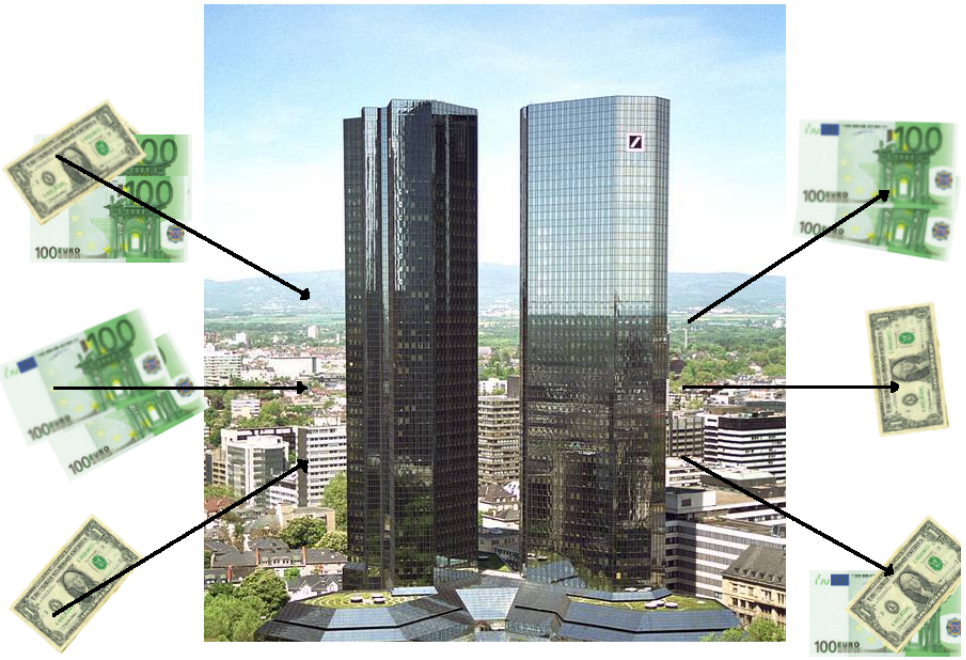
David Chaum's mix (1981) and cascades of mixes are the traditional basis:



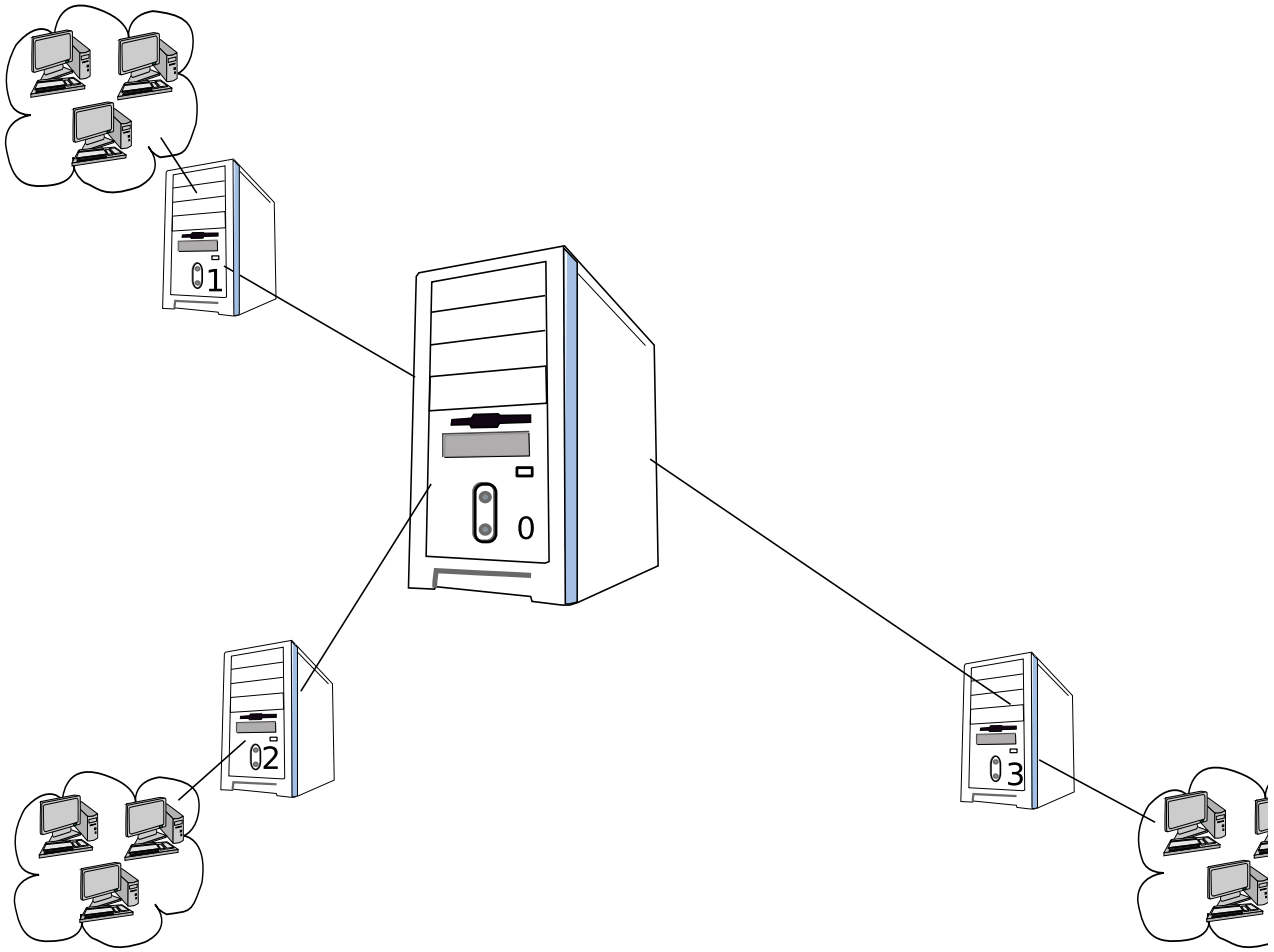
The Traditional Approach

David Chaum's mix (1981) and cascades of mixes are the traditional basis:

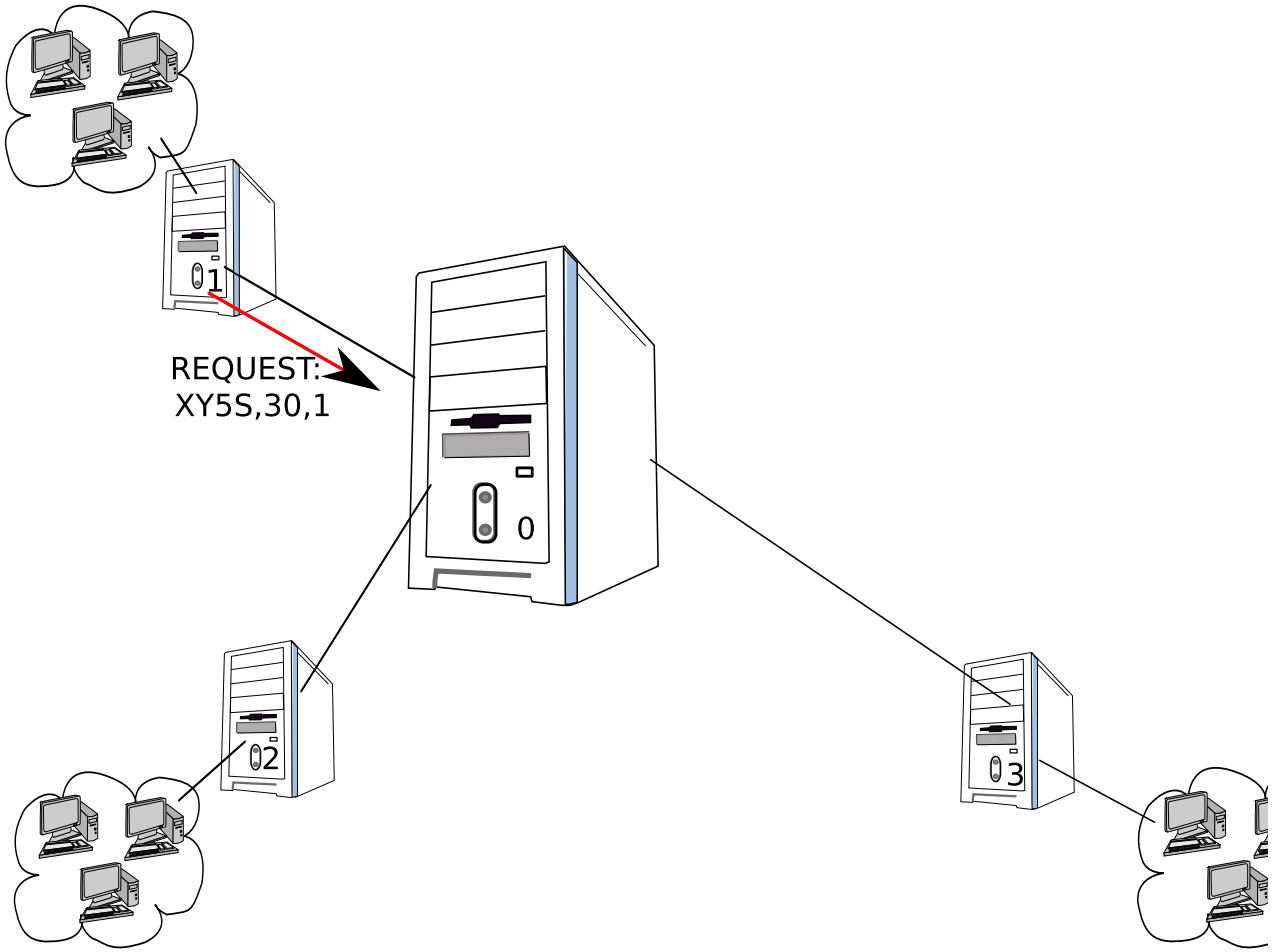
Money Laundering



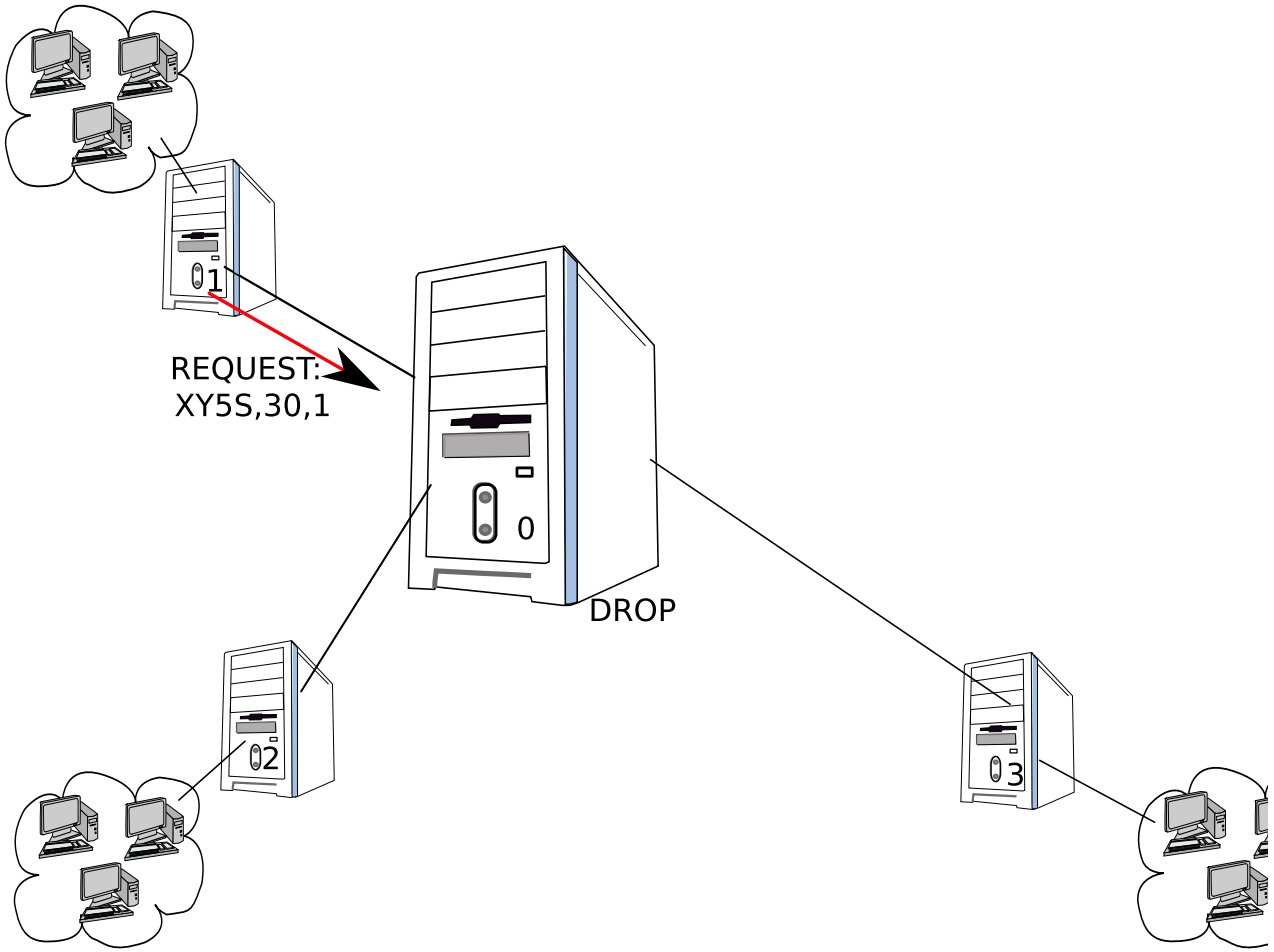
GAP illustrated (1/9)



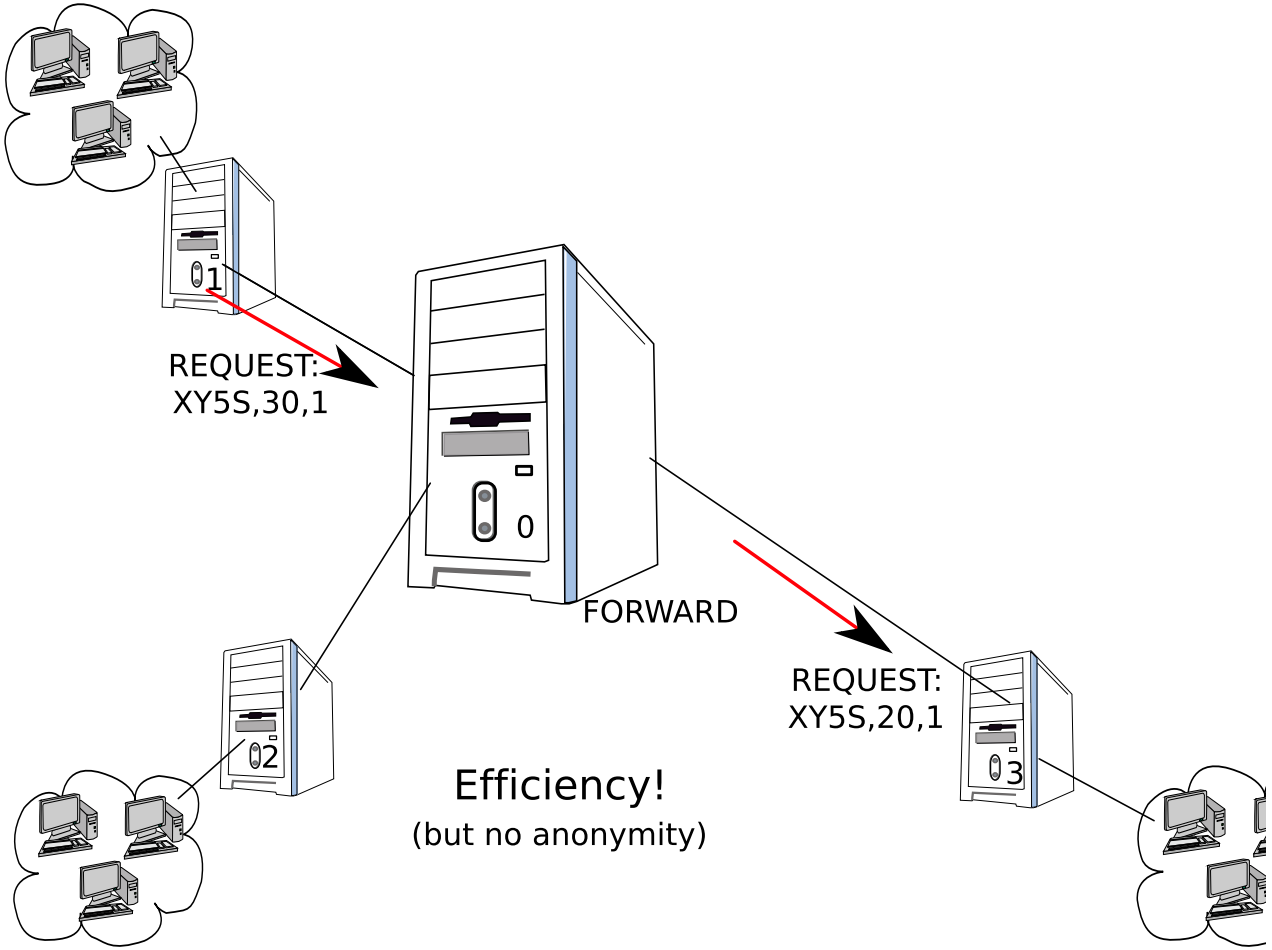
GAP illustrated (2/9)



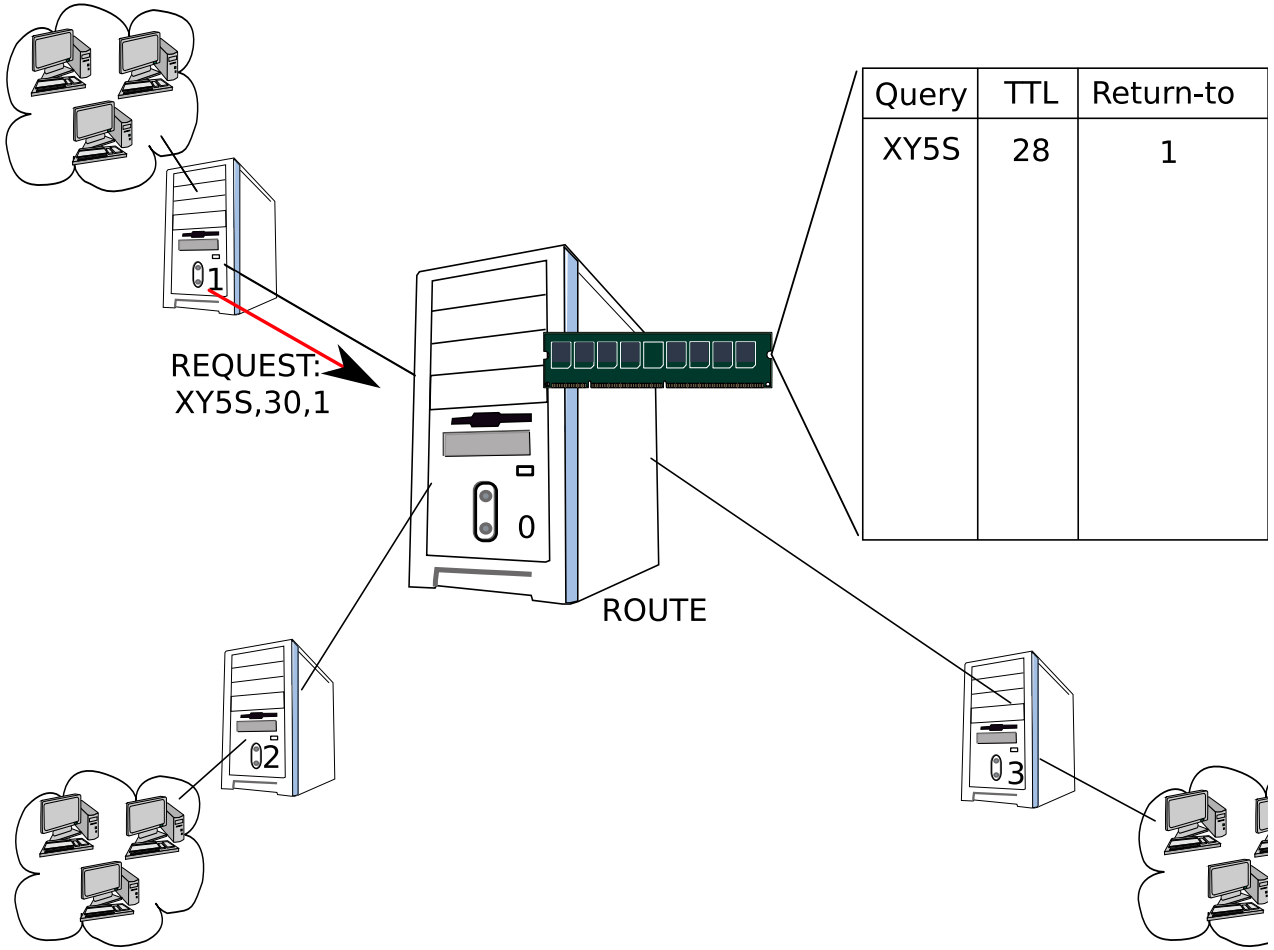
GAP illustrated (3/9)



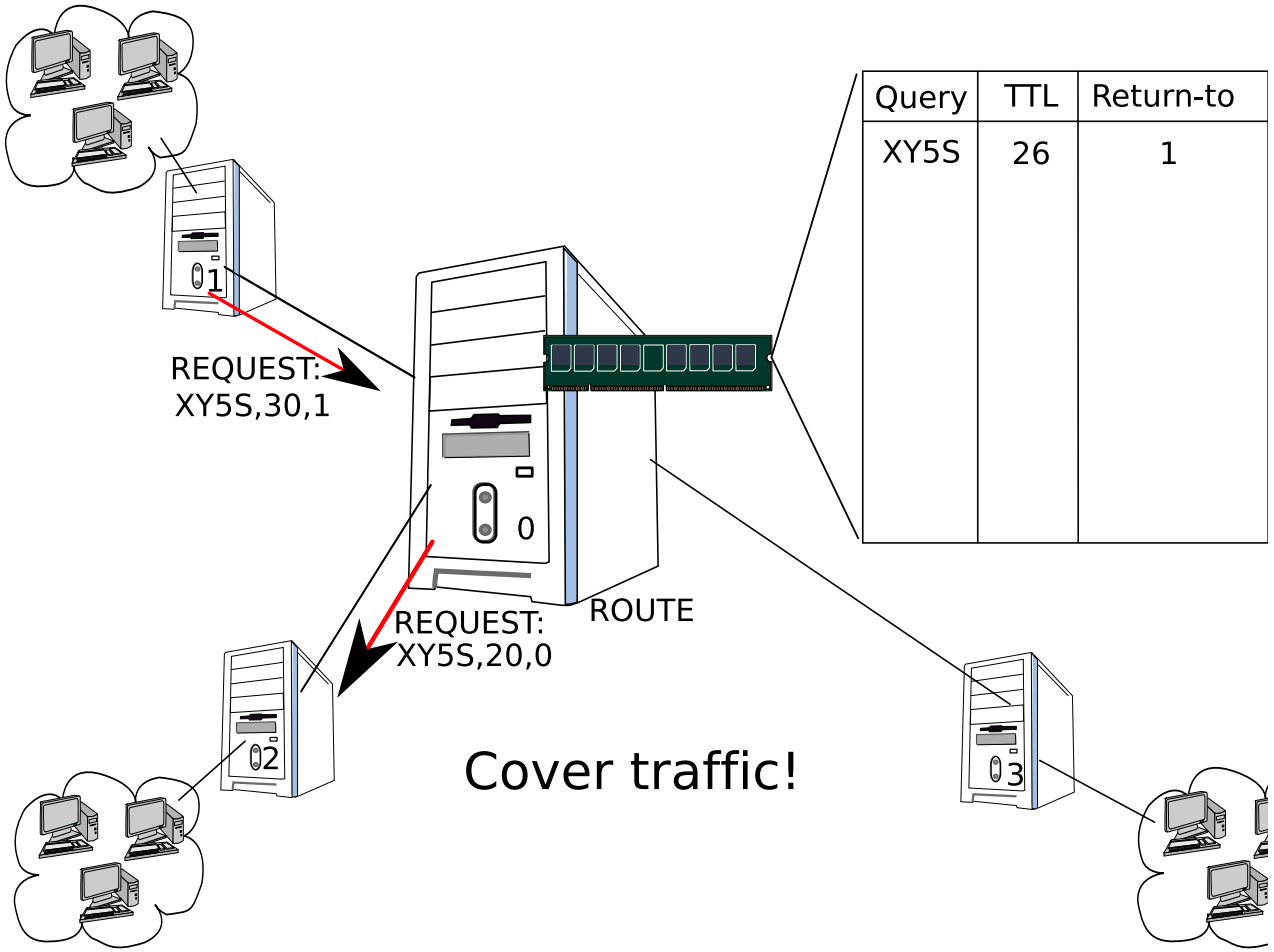
GAP illustrated (4/9)



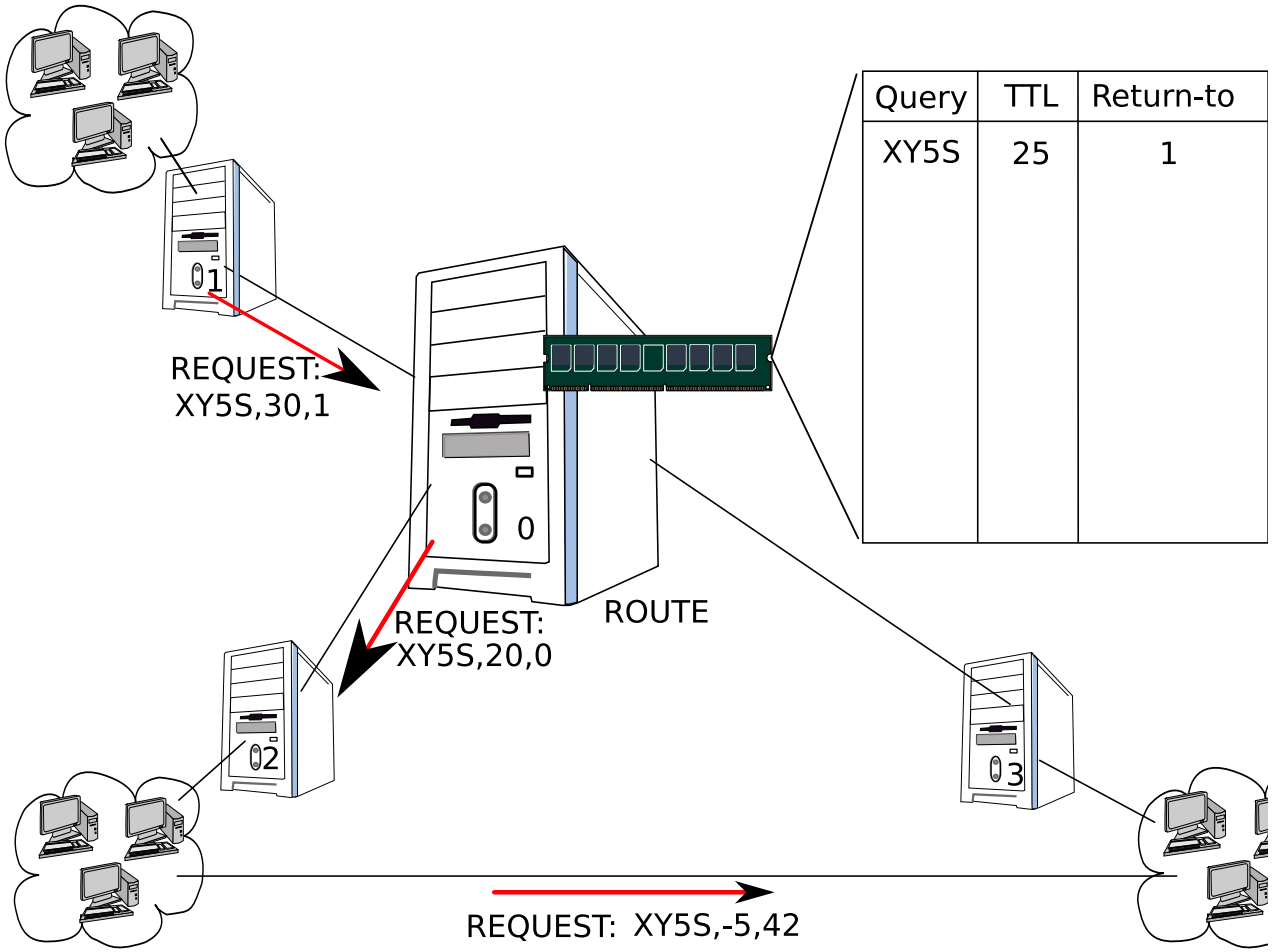
GAP illustrated (5/9)



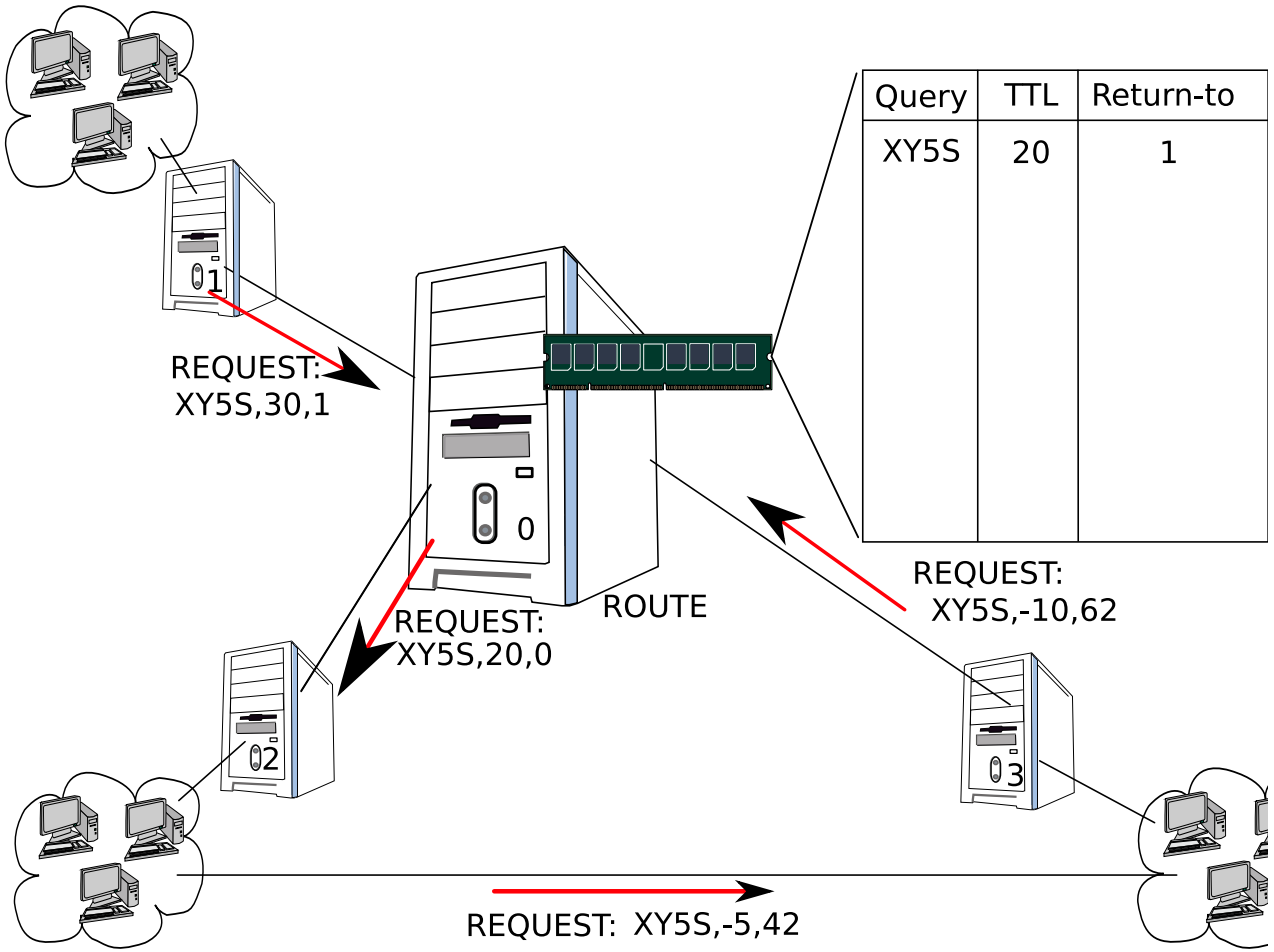
GAP illustrated (6/9)



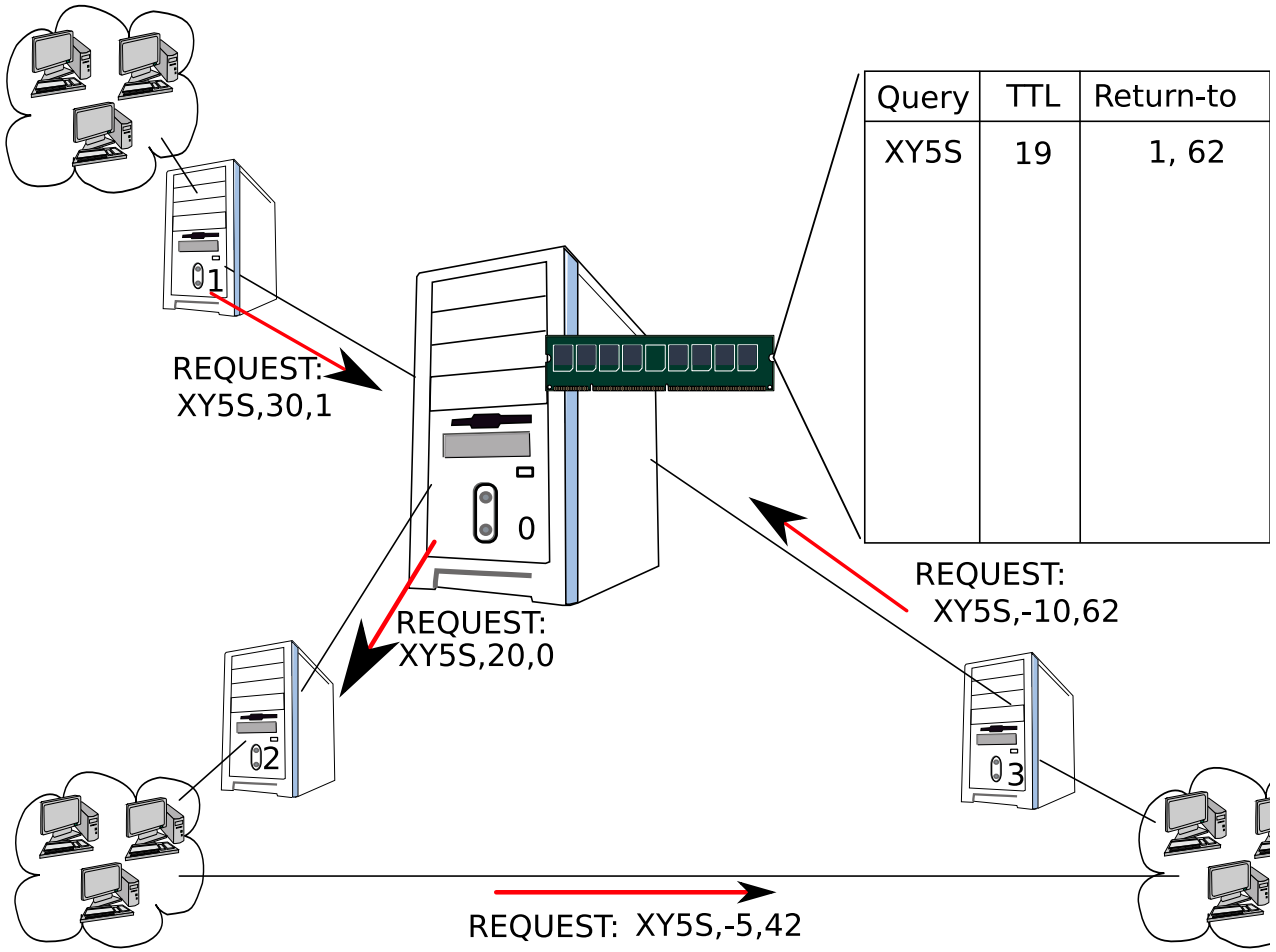
GAP illustrated (7/9)



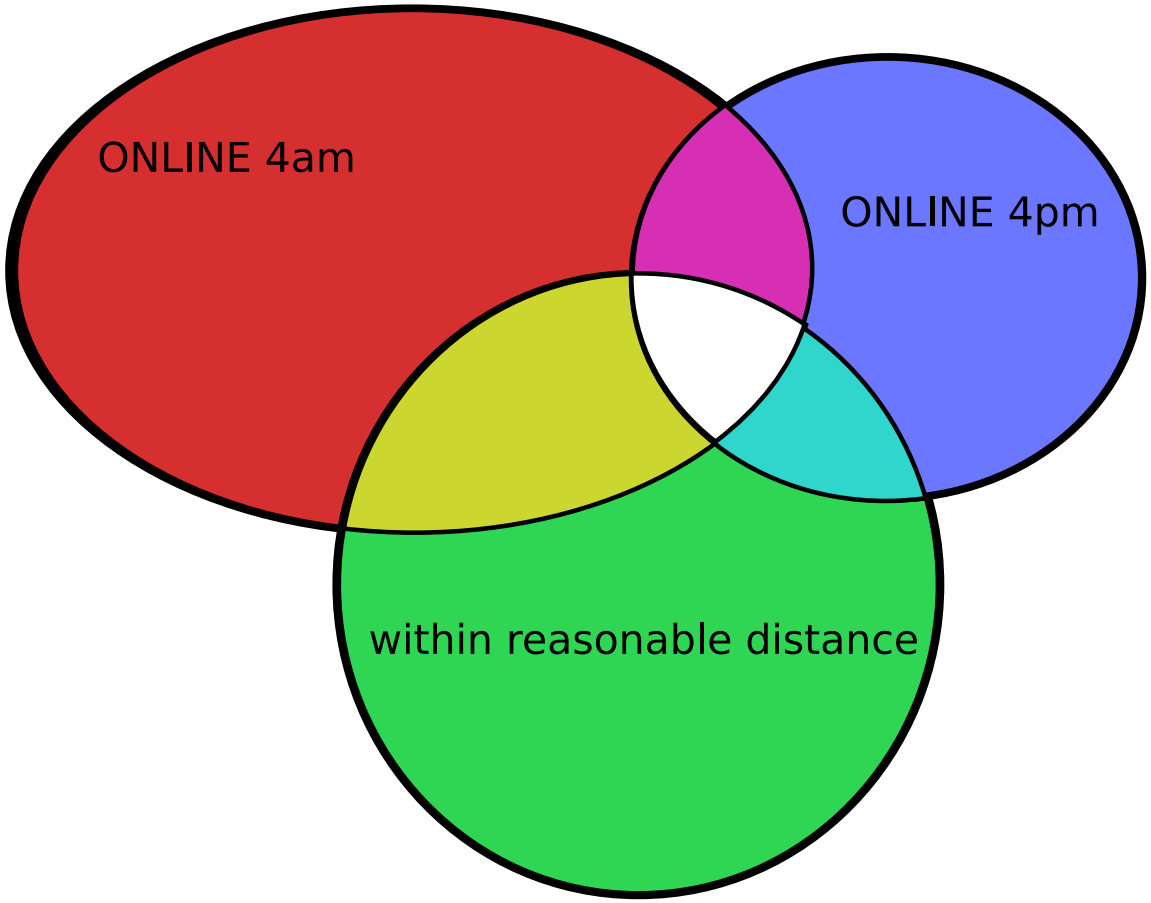
GAP illustrated (8/9)



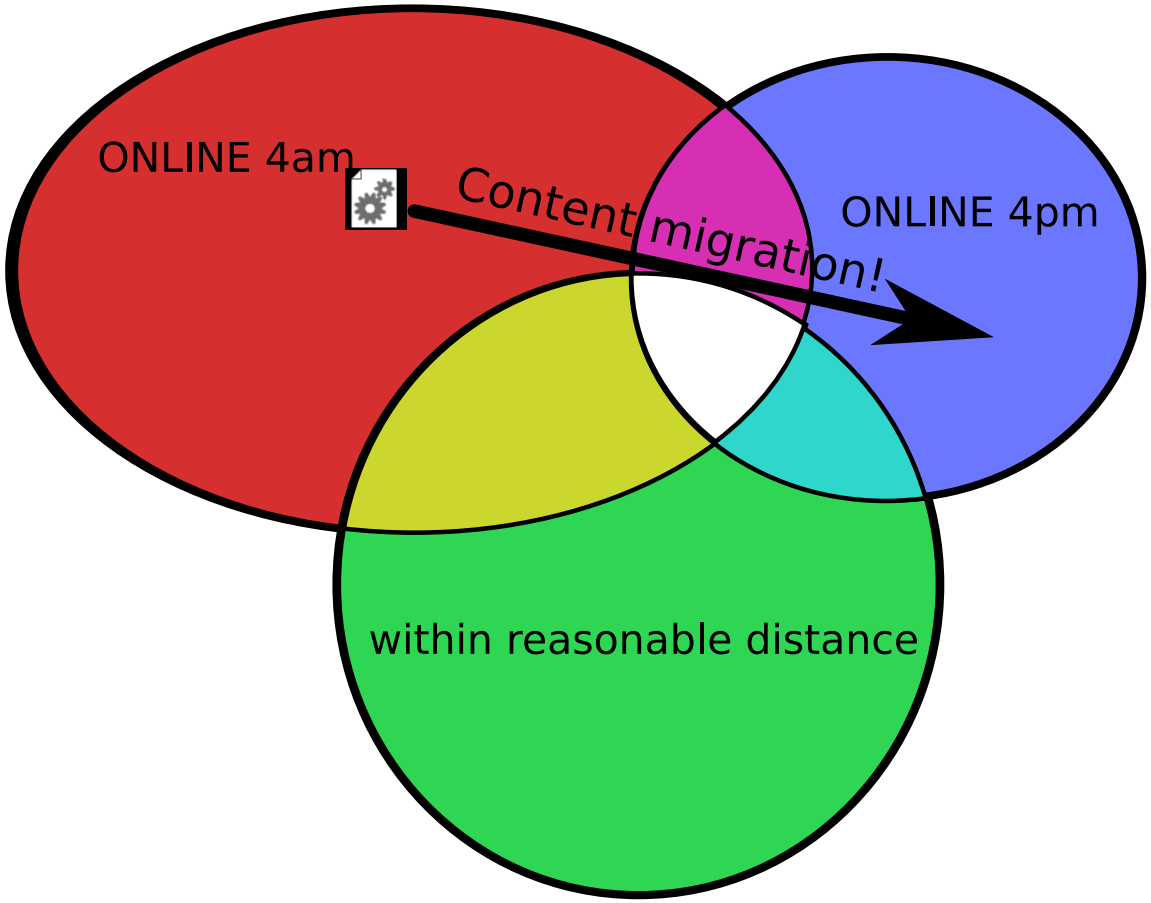
GAP illustrated (9/9)



Attacks: Partitioning (1/2)



Attacks: Partitioning (2/2)



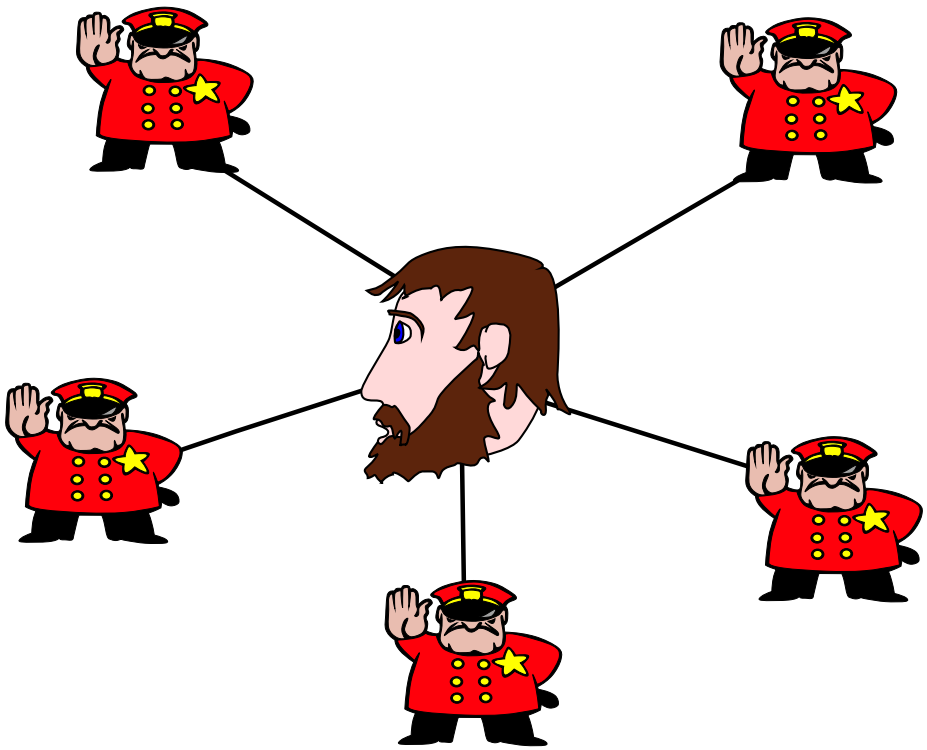
Attacks: Probabilistic

- GAP tries to equalize probabilities:
  - Highly connected, link-encrypted network
  - Hot-path routing with random delays
  - Query and content padding in addition to noise
  - Content encoding makes correlation difficult; uniform message sizes

  ... but strong adversary may still succeed

$\Rightarrow$ configure to require more cover traffic

Attacks: Isolation



Anonymity: Conclusion

It is possible for an anonymous network to

- Allow users to choose individual trade-offs
- Make indirection of replies optional
- Control anonymous routing with a TTL

Open research question: Statistical attacks & performance?

Conclusion

GNUnet combines features in a decentralized P2P network that were previously considered mutually exclusive:

- Anonymity
- Deniability
- Decentralization
- Economic incentives

Current Work

Funded by **DFG ENP GR 3688/1-1** since 9'2009:[1]

- Non-anonymous routing (a question of scale)
- Automatic transport selection (network neutrality coding)
- New applications

To facilitate this, we are making some **major architectural changes**.

[1] I'm still trying to hire additional PhD students.

Architectural Changes for 0.9.x (1/7)

0.8.x `gnunetd` daemon process with many threads loading many *plugins* to support half a dozen different applications

- Deadlocks, data races, 1,000 lines with locking operations
- Memory corruption bugs not isolated between plugins
  $\Rightarrow$ not scalable

0.9.x `gnunet-arm` coordinates many single-threaded processes communicating via `loopback`

Architectural Changes for 0.9.x (2/7)

0.8.x `gnunetd` daemon process with central configuration and basic management functions

- Configuration changes almost always required full `gnunetd` restart
- Virtually impossible to write plugins in anything but C/C++

0.9.x `gnunet-arm` restarts components as needed, written in any language

Architectural Changes for 0.9.x (3/7)

0.8.x Basic APIs (`libgnunetutil`) had some design issues:

- Relative and absolute time not distinguished
- Time could be in seconds or milliseconds
- Use of logging facilities resulted in rather verbose code

0.9.x New, cleaner APIs eliminating entire categories of common usage bugs

Architectural Changes for 0.9.x (4/7)

0.8.x Statistics service provided easy way to inspect system status, but lacked:

- API for processes outside of `gnunetd` to set/update values
- No persistence of values across `gnunetd` restarts
  $\Rightarrow$ replication of peristence functions for simple numbers

0.9.x Persistent statistics with uniform API (read, update)

Architectural Changes for 0.9.x (5/7)

0.8.x Deep, complex directory hierarchy, often one directory per module

- Coverage evaluation with `lcov` not possible
- Directories with very few files
- Many helper libraries

0.9.x Codebase restructured, clear naming conventions, flat hierarchy

Architectural Changes for 0.9.x (6/7)

0.8.x Peers introduce each other using HELLO messages, one per transport

- Each transport could only have one IP address
- Multiple transports each required (relatively large) HELLO messages
- User had to configure IP addresses, other peers could not help with discovery

0.9.x One HELLO per peer with many addresses, can learn IPs from other peers

Architectural Changes for 0.9.x (7/7)

0.8.x Crash in file-sharing lost all activities

0.9.x Fine-grained serialization of file-sharing state for loss-less recovery

Current & Near-Future Work

- Finish core/fs rearchitecturing
- GNUnet over WiFi (obsolete ISPs)
- IPv6 over GNUnet (Tor & more)
- GNUnet monkey: Automatic debugging using GDBMI
- UPnP support

GNU helps GNUnet? (1/2)

- `GtkFileChooser` "save as" dialog does not allow me to specify a default file name (for a file that does not yet exist)
- GNU libmicrohttpd does not do SSL properly, GNUtls APIs do not seem to support certain event handling methods (select-based, thread pool)
- ECC support in `libgcrypt`?

GNU helps GNUnet? (2/2)

- GNU social (not yet a GNU package?) never talked to us, but there is Snag (Social Networking Application for GNUnet)
- Advertising PhD positions on `fsf.org` failed – alternatives?
- GNU libextractor requires parsers for more file formats, please contribute!

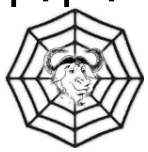
References

[1] Krista Bennett, Christian Grothoff, Tzvetan Horozov, and J. T. Lindgren. An Encoding for Censorship-Resistant Sharing. Technical report, 2003. available at https://gnunet.org/ecrs.

[2] Friedrich Engels. Umrisse zu einer Kritik der Nationalökonomie. 1844.

[3] Christian Grothoff. An Excess-Based Economic Model for Resource Allocation in Peer-to-Peer Networks. Wirtschaftsinformatik, 3-2003, June 2003.

[4] Marc Waldman, Aviel D. Rubin, and Lorrie Faith Cranor. Publius:



A robust, tamper-evident, censorship-resistant, web publishing system. In Proc. 9th USENIX Security Symposium, pages 59–72, August 2000.

RTFL

GNUnet is a GNU package.